

Reviewer Pack — Minimal Monodromy Group Order and Resolution Proof Width Cor...

Entry 92c1f84825b0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 16:07:03 UTC

Minimal Monodromy Group Order and Resolution Proof Width Correlation

Entry ID: 92c1f84825b0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 16:07:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monodromy Groups) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Boolean satisfiability problem (SAT) instance with m clauses and n variables, the minimal order of the monodromy group associated with its Tseitin transform is linearly correlated with its resolution proof width, such that $|\text{Mon}(G)| \leq \alpha w(\varphi)$, where G is the Tseitin graph, φ is the associated formula, $w(\varphi)$ is the resolution proof width, and α is a constant.

Rationale (proposer's reasoning):

The study of monodromy groups in algebraic geometry has revealed deep connections to symmetry and structure in mathematical objects. By linking this concept with resolution proof complexity, we may uncover new insights into the intrinsic complexity of SAT instances. The constructiveness of the mapping from SAT instances to monodromy groups allows for a direct computational verification of the conjecture.

Taxonomy category: Algebraic Geometry - Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 44c038eb0ff15f4b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 90% of the computed monodromy group orders are within a linear correlation with resolution proof width, such that $|\text{Mon}(G)| \leq \alpha w(\varphi)$ for all instances, where α is calculated based on the data. The criterion is falsified if less than 70% of the computed monodromy group orders are within this correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monodromy group order" AND "resolution proof width" - "Tseitin transform" AND "algebraic geometry" - "Boolean satisfiability problem" AND "minimal monodromy group order"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n, m):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for _ in range(m):
            literals = random.sample(variables + [f'~{v}' for v in variables], 2)
            clauses.append(f"({' | '.join(literals)})")
        return clauses

    def tseitin_transform(clauses):
        new_vars = {}
        new_clauses = []
        var_count = 1
        for clause in clauses:
            literals = [l.strip('~') for l in clause.split(' | ')]
            if len(literals) == 2 and literals[0] != literals[1]:
                tseitin_var = f'y{var_count}'
                new_vars[tseitin_var] = var_count
                new_clauses.append(f"({literals[0]} -> {tseitin_var})")
                new_clauses.append(f"({literals[1]} -> {tseitin_var})")
                new_clauses.append(f"(~{tseitin_var} -> ~{literals[0]})")
                new_clauses.append(f"(~{tseitin_var} -> ~{literals[1]})")
                var_count += 1
            else:
```

```

        new_clauses.append(clause)
    return new_clauses, new_vars

def resolution_prove(clauses):
    clauses_set = set(clauses)
    unit_clauses = [c for c in clauses if ' | ' not in c]
    while unit_clauses:
        unit_clause = unit_clauses.pop()
        literal = unit_clause.strip('~')
        polarity = 1 if literal[0] != '~' else -1
        new_clauses = []
        for clause in clauses_set:
            if literal in clause:
                continue
            if f"~{literal}" in clause:
                new_clauses.append(clause.replace(f"~{literal}", ""))
            else:
                new_clauses.append(clause)
        unit_clauses.extend([c for c in new_clauses if ' | ' not in c])
        clauses_set.update(new_clauses)
    return len(clauses_set)

def monodromy_group_order(n):
    # Placeholder implementation
    # This should be replaced with a proper algorithm to compute the monodromy group order
    return n + 1

results = []
for n in [5, 10, 15, 20, 30, 40]:
    m = random.randint(2 * n, 4 * n)
    sat_instance = generate_sat_instance(n, m)
    tseitin_clauses, new_vars = tseitin_transform(sat_instance)
    proof_width = resolution_prove(tseitin_clauses)
    monodromy_order = monodromy_group_order(n)
    results.append((monodromy_order, proof_width))

if not results:
    return {
        "metric_name": "Monodromy Group Order vs Resolution Proof Width",
        "metric_value": 0.0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

monodromy_orders = [r[0] for r in results]
proof_widths = [r[1] for r in results]
alpha = sum(monodromy_orders) / sum(proof_widths)
correlation = sum((m - alpha * w) ** 2 for m, w in zip(monodromy_orders, proof_widths)) / len

return {
    "metric_name": "Monodromy Group Order vs Resolution Proof Width",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n for n in [5, 10, 15, 20, 30, 40]),

```

```

        "conjecture_holds": correlation <= 0.9 * alpha,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction < 0.7:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{first_failing_seed}' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20805
2	propose	ollama_remote	glm4:latest	0	0	12857
3	preregistration	ollama_remote	glm4:latest	0	0	14639
4	novelty	ollama_remote	glm4:latest	0	0	8334
5	novelty	ollama_remote	glm4:latest	0	0	9368
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23294
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12334
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13832
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13921
10	judge	ollama_remote	glm4:latest	0	0	36633

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 166017 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/92c1f84825b0.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/92c1f84825b0.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/92c1f84825b0.tar.gz* (if generated)